

10 · 无 GPU 后训练情况下的优化策略

2026-07-31

Contents

10 · 无 GPU 后训练情况下的优化策略	2
1. 约束与思路	2
2. 策略矩阵	2
3. Prompt Engineering	2
3.1 角色化系统提示	2
3.2 结构化分段	2
3.3 Few-shot 示例	2
3.4 负例约束	3
3.5 思维链引导	3
3.6 效果评估	3
4. Multi-Agent 分工	3
5. RAG 定理检索	3
5.1 实现	3
5.2 收益	3
5.3 检索质量优化	3
6. Tool Calling	3
6.1 工具集	3
6.2 闭环价值	3
6.3 实现要点	4
7. Verification	4
8. Self-Reflection	4
8.1 触发	4
8.2 流程	4
8.3 收益	4
8.4 实现	4
9. Self-Consistency Voting	4
9.1 实现	4
9.2 收益	4
9.3 成本控制	5
10. 程序化数据合成	5
11. 轻量微调 (小模型)	5
11.1 范围	5
11.2 成本	5
11.3 收益	5
12. 策略组合与协同	5

12.1 组合效果	6
13. 效果评估与迭代	6
14. 设计路线小结	6

10 · 无 GPU 后训练情况下的优化策略

本文档阐述在无法进行大规模后训练的约束下，通过非训练手段提升系统效果的策略矩阵，各策略的实现方法、效果评估与组合方式。

1. 约束与思路

团队不具备大规模后训练的 GPU 资源，因此系统效果提升完全依赖**工程与算法策略**而非模型训练。核心思路：**以工程换精度**——通过 Prompt、Agent、RAG、Tool Calling、Verification、Reflection、Voting 等手段，将开源 LLM 的能力压榨到极致，并用符号系统保证严格性。

2. 策略矩阵

策略	作用环节	收益	成本	详见
Prompt Engineering	推理	直接提升 LLM 几何推理质量	低	§ 3
Multi-Agent 分工	感知/关系	降低单模型负担，提升准确率	中	§ 4
RAG 定理检索	推理	注入领域知识，减少幻觉	中	§ 5
Tool Calling	推理	LLM 调用 Verifier/Solver，闭环验证	中	§ 6
Verification	全链路	阻断错误假设，保证严格性	中	§ 7
Self-Reflection	推理	失败回溯，自我修正	中	§ 8
Self-Consistency Voting	推理	多路径投票，提升稳健性	高	§ 9
程序化数据合成	数据	零成本海量训练/评估数据	低	§ 10
轻量微调（小模型）	感知	标注符号检测，单卡可训	低	§ 11

3. Prompt Engineering

3.1 角色化系统提示

明确”几何专家 + 仅基于已验证事实推理 + 不得跳步”。

3.2 结构化分段

DSL / 已验证关系 / 定理 / 任务 / 指令清晰分段，降低 LLM 注意力分散。

3.3 Few-shot 示例

提供 2~3 个高质量证明范例（按题型动态选取），覆盖典型证明模式。

3.4 负例约束

明确”不得使用未验证关系” “不得编造数值” “verify 失败必须 reflect”。

3.5 思维链引导

要求 LLM “先列已知与目标 → 再规划 → 再验证 → 再汇总”。

3.6 效果评估

用合成题集对比不同 Prompt 的准确率，A/B 选优。Prompt 版本化，回归测试。

4. Multi-Agent 分工

详见 04-Relation-Agents 与 01-Parser。- 感知层异构检测器并行。- 关系层按对象类型分 Agent。- 收益：上下文聚焦、错误可定位、可并行、可扩展。

5. RAG 定理检索

5.1 实现

- 定理库向量化 (FAISS/Chroma)。
- 按 DSL 对象/关系检索 Top-K 相关定理。
- 注入 Prompt [Context]。

5.2 收益

- 注入领域知识，减少 LLM 幻觉（如记错圆周角定理）。
- 提示证明思路（检索到”相似对应边成比例”引导相似路径）。

5.3 检索质量优化

- 混合检索：向量 + 关键词（定理名）。
 - 重排序：按对象类型匹配度重排。
 - 失败回退：检索为空时，LLM 自行回忆（但标记低置信）。
-

6. Tool Calling

6.1 工具集

verify / solve / search / reflect / graph_query / construct（见 07-LLM-Agent § 4）。

6.2 闭环价值

- 每个中间结论可验证，把 LLM”自由发挥”约束在”可计算”轨道。
- Solver 提供精确计算，LLM 不需也不允许编造数值。
- 这是抑制幻觉的最有效手段。

6.3 实现要点

- 工具结果结构化 JSON，LLM 易解析。
 - 失败结果含 reason，供 reflection。
 - 工具调用次数限制（如 30 次），防死循环。
-

7. Verification

详见 05-Verifier。- 三态输出 (true/false/uncertain)。- 自适应容差。- 阻断错误假设进入证明链。- 是”可验证”原则的基石。

8. Self-Reflection

8.1 触发

Verifier/Solver 返回 false/失败时。

8.2 流程

1. 分析失败原因（关系不存在 / 定理用错 / 规划偏差）。
2. 生成修正规划。
3. 重新尝试（限制 3 轮）。

8.3 收益

- 自我修正，提升复杂题通过率。
- 避免在死胡同浪费算力。

8.4 实现

```
def reflect(failure, plan, history):  
    return llm(f"""  
    失败: {failure}  
    规划: {plan}  
    历史: {history}  
    分析原因并修正规划。最多 3 轮。  
    """)
```

9. Self-Consistency Voting

9.1 实现

对同一题，不同温度采样 N（如 5）条独立证明路径，对答案投票（见 07 § 3.4）。

9.2 收益

- 多数投票抑制单路径错误。

- 分歧检测触发更深搜索或人工复核。

9.3 成本控制

- N=5 路径, $5 \times$ 推理成本。
 - 仅对高 stakes 题或首轮分歧题启用。
 - 可用更小模型做初筛, 大模型做精证。
-

10. 程序化数据合成

详见 09-Dataset § 4。 - 零成本海量数据。 - 用于训练轻量检测器、评估各模块、构建 Few-shot。

11. 轻量微调 (小模型)

11.1 范围

仅用于感知层标注符号检测 (YOLOv8-n) 与可选关键点网络 (MobileNetV3-small)。

11.2 成本

- 数百张标注 + 单卡数小时。
- 模型小 ($< 10M$ 参数), 推理快。
- 契合”不依赖大规模后训练”原则。

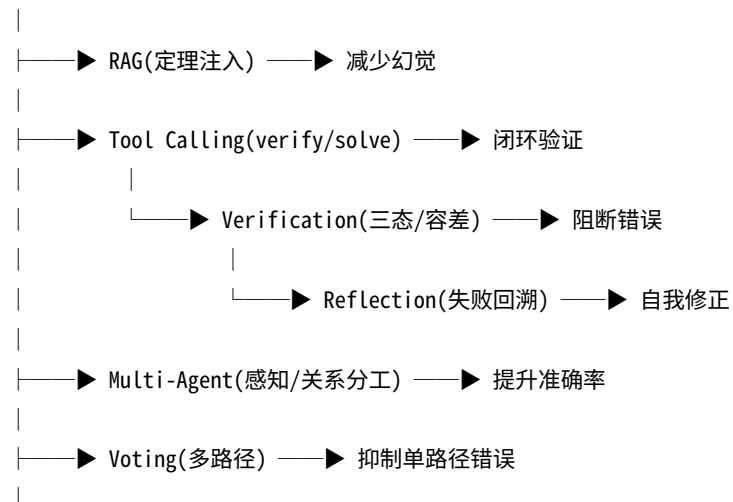
11.3 收益

- 标注符号检测准确率显著高于传统 CV。
 - 关键点网络补低质量图召回。
-

12. 策略组合与协同

各策略非独立, 形成协同闭环:

Prompt(角色/结构/Few-shot/负例)



└──► 合成数据 + 轻量微调 ──► 感知层支撑

12.1 组合效果

单策略收益有限，组合后形成”感知精确 + 推理受约束 + 验证阻断 + 反思修正 + 投票稳健”的鲁棒体系，在不训练大模型的前提下逼近后训练效果。

13. 效果评估与迭代

指标	评估方式
端到端答案准确率	真题评估集
证明正确率	Solver/Lean 校验通过率
幻觉率	未验证关系进入结论的比例
反思触发率	失败回溯频率
投票一致性	多路径答案一致率

迭代：每策略独立 A/B，正向叠加；回归测试防回退。

14. 设计路线小结

无后训练优化策略的设计路线为：“Prompt 打底 → RAG 注入知识 → Tool Calling 闭环 → Verification 阻断 → Reflection 修正 → Voting 稳健 → 合成数据 + 轻量微调支撑感知”。七策略协同，以工程换精度，在零大模型训练成本下构建高准确率、可解释、可验证的几何求解能力。